

Curación interna — veredicto sobre las 48

Verificado contra Mathlib4, commit `17019dc` (14 sep 2026). El veredicto de las 172 citaba el árbol `05322f9` (28 ago), así que la pregunta de caducidad tiene respuesta exacta: diecisiete días de diferencia, y se nota en una ficha. El `.log` que acompaña al PDF es el registro de pdfTeX de la compilación; no trae datos de curación.

El diagnóstico, uno por montón

A no son diez búsquedas. Una decisión ha caducado, dos no caducan nunca y por razones distintas, una no se resuelve buscando sino fusionando, y seis aguantan. Solo una ficha añade nombre, que es justo lo que querías saber antes de pagar el coste de A.

B no son treinta y seis decisiones, pero tampoco es una marca nueva. Lo que falta es una distinción: el grafo tiene dos clases de nodo —concepto y rama— y una sola columna para las dos. La solución es un campo aparte, `rol`, no una sexta marca en la columna de `marca`. Si R entra como marca, se pierde el veredicto T, que es exactamente lo que impide que mañana estos nodos tomen nombres; y la auditoría que produjo esta hoja —marca T con hijos— deja de poder ejecutarse, porque ya no habría contradicción que detectar. Además la capa ya existe: tienes 22 áreas sin marca «que no son suyas». Los 36 son ramas que acabaron en la capa de nodos.

C no es «una de las dos mitades sobra». Es que el campo `lean` hace dos trabajos: identidad ontológica del vértice y vocabulario para el prompt. Son cosas distintas y por eso pueden contradecirse. En una ficha gana una mitad y en la otra gana la otra, y separar el campo evita que vuelva a pasar.

Montón A · las diez

nodo	decisión	evidencia en el árbol de hoy
brownian-motion	ya no es verdad → entra <code>ProbabilityTheory.IsBrownianReal</code>	<code>Probability/BrownianMotion/Basic.lean:302</code> , con <code>IsPreBrownianReal (:75)</code> y <code>BrownianMotion/GaussianProjectiveFamily.lean</code>
sequent-calculus	no se decide buscando → se retira por fusión con <code>fol-deduction</code>	ya estaba en la lista de vértices duplicados; 3 hijos, 0 sueltos, así que la retirada es gratis
homotopy-type-theory	aguanta, y no caduca nunca	<code>Eq : Prop</code> con irrelevancia definicional; es fundamento, no biblioteca
cic	aguanta, y no caduca por crecimiento	Mathlib no va a declarar el CIC porque es el metalenguaje en que está escrito
lambda-calculus	aguanta, pero sí puede caducar	hoy no hay nada; a diferencia del CIC, el cálculo lambda sí es formalizable como objeto dentro de Lean
homotopy-theory	aguanta, por poco	el marco creció mucho (21 ficheros en <code>AlgebraicTopology/ModelCategory</code> : cilindros, objetos de caminos, lema de Brown), pero las únicas instancias de <code>ModelCategory</code> siguen siendo <code>CochainComplex.Plus, Over S y C^{op}</code> : ninguna sobre <code>SSet</code> ni <code>TopCat</code>
planar-graphs	aguanta	no hay planaridad; <code>Coplanar (LinearAlgebra/AffineSpace/FiniteDimensional.lean:731)</code> es geometría afin, falso amigo
geometric-topology	aguanta	no hay nudos ni 3-variedades; lo más cercano es <code>Algebra/Quandle.lean</code> , que es un invariante algebraico, no una variedad
symplectic-geometry	aguanta	solo <code>symplecticGroup</code> y <code>symJ (LinearAlgebra/SymplecticGroup.lean)</code> ; no hay forma ni variedad simpléctica, que es lo que promete la marca C
fol-deduction	aguanta	<code>ModelTheory/</code> tiene <code>Syntax, Semantics, Satisfiability</code> ; no hay <code>inductive Proof</code> ni <code>⊢</code> sintáctico

Dos observaciones que salen de la tabla y no de las fichas:

cic y lambda-calculus no caducan igual. Los dos llevan hoy el mismo `lean=None`, pero uno es imposible por construcción —el CIC es aquello en lo que Mathlib está escrito, no algo que Mathlib pueda declarar— y el otro es un hueco de biblioteca como cualquier otro. Si el campo de caducidad distingue «nunca» de «todavía no», `cic` se va con `homotopy-type-theory` y deja de volver a la hoja.

homotopy-theory aguanta, pero su vecindario se movió. Ahora existe `AlgebraicTopology/SingularHomology/`, con `singularChainComplexFunctor` y `singularHomologyFunctor : C ⇔ TopCat ⇔ C (Basic.lean:36 y :52)`. Eso no le da nombre a este vértice —sigue sin localización—, pero sí se lo da a la arista `homology` que sale de él y toca `algebraic-topology`. Es un nombre para la capa de flechas, y ahí es donde hay que apuntarlo.

Montón B · las treinta y seis

La regla, que vale para las treinta y seis y es auditable: **el objeto que encuentre el índice nunca entra en el nodo-rama.** Si es C, S u O, es un hijo nuevo; si es F, es evidencia. Un nodo-rama no toma nombre nunca, y por eso puede quedarse con su T sin contradicción.

nodo	hijos · sueltos	decisión
zfc-axioms	143 · 20	a la capa de áreas: con 143 hijos no es un concepto, es la raíz fundacional. Si prefieres mantenerlo como nodo, entonces la marca honesta es C con <code>ZFSet (SetTheory/ZFC/Basic.lean)</code> , porque el universo V sí es categoría —pero el token <code>set</code> es genérico y hay que declararle keyword estrecha
lean-kernel	10 · 1	a la capa de áreas: mismo caso, raíz del subárbol del metalenguaje
recursion-theory	5 · 0	fusión con <code>computability-theory</code> : sinónimo, ya estaba en la lista de duplicados, 0 sueltos
computability-theory	7 · 2	rama · hijo nuevo <code>turing-degrees</code>
extremal-combinatorics	3 · 0	rama · hijo nuevo <code>extremal-graphs</code>
matching-theory	3 · 0	rama · hijo nuevo <code>matchings</code>

contour-integration	4 · 1	rama · evidencia <code>circleIntegral</code> (es un operador: F, no vértice)
quadratic-residues	3 · 0	rama · evidencia <code>legendreSym</code> (es un carácter: F)
yoneda-lemma	3 · 0	rama · evidencia <code>CategoryTheory.yoneda</code> (es un funtor: F)
prime-factorization	3 · 0	rama · el objeto ya tiene nodo propio, <code>unique-factorization</code>
proof-theory	6 · 2	rama · su objeto es el vértice <code>fol-deduction</code> , que es su hijo
algebraic-combinatorics	4 · 0	rama · tanda de retirada medida (1 palabra clave)
probabilistic-method	3 · 0	rama · tanda de retirada medida (1 palabra clave)
ramsey-theory	3 · 0	rama · tanda de retirada medida (1 palabra clave)
analytic-number-theory	6 · 1	rama · tanda de retirada medida (1 palabra clave)
discrete-optimization	1 · 0	rama · tanda de retirada medida (1 hijo: no enruta, encadena)
linear-programming	1 · 0	rama · tanda de retirada medida (1 hijo)
variational-methods	1 · 0	rama · tanda de retirada medida (1 hijo)
canonical-forms, pde-techniques, residue-theorem, universal-properties, enumerative-combinatorics, inclusion-exclusion, algorithm-analysis, computational-complexity, formal-verification, np-completeness, circle-geometry, compactness-theorem, incompleteness, diophantine-equations, prime-number-theorem, convex-optimization, limit-theorems, forcing	—	rama declarada, sin más

Total: 2 a la capa de áreas, 1 fusión, 33 ramas declaradas.

Los tres objetos que sí esperan

padre	hijo nuevo	marca	nombre
computability-theory	turing-degrees	C (orden parcial)	<code>TuringDegree · Computability/TuringDegree.lean</code>
extremal-combinatorics	extremal-graphs	S	<code>SimpleGraph.IsExtremal, SimpleGraph.extremalNumber · Combinatorics/SimpleGraph/Extremal/Basic.lean</code>
matching-theory	matchings	S (subobjetos)	<code>SimpleGraph.Subgraph.IsMatching · Combinatorics/SimpleGraph/Matching.lean</code>

`turing-degrees` es además lo que estaba sosteniendo a `recursion-theory`: al fusionarla, el objeto que el índice encontró bajo ella no se pierde, baja un nivel y queda donde debía estar.

La tanda de retirada medida

Siete nodos —cuatro con una sola palabra clave y tres con un solo hijo— son los únicos donde «rama» es discutible, porque una rama con una palabra no enruta y una rama con un hijo encadena en vez de ramificar. Tu propia asimetría los hace baratos: quitar no baja la precisión, así que van en una sola tanda. Si los bancos no se mueven, se retiran; si baja su barrio, estaban enrutando y se quedan. Es la única parte de B que necesita medición.

Montón C · las dos

`limits-continuity`: gana la marca, y los nombres se mueven. La marca T está bien —continuidad es ser morfismo y límite es una construcción: las dos son capa de flechas—, pero `Continuous`, `ContinuousAt` y `Filter.Tendsto` son vocabulario correcto aunque no sean identidad de un vértice. No se borran: bajan al campo de evidencia declarada, que es donde sirven sin prometer nada. Y es comprobable barato: quítalos del prompt y mira su barrio; si no baja, tampoco estaban aportando.

`cardinal-arithmetic`: gana la voz, y la marca cambia. Aquí el nodo está mal nombrado, no mal marcado. La aritmética son operaciones —eso sí es T— pero `Cardinal` es un objeto con 645 citas (`SetTheory.Ordinal.Univ.lean`), y el propio veredicto ya decía que el sustrato es categoría delgada. El nodo pasa a llamarse `cardinals`, marca C, objetos los cardinales y flechas $\leq$, con `Cardinal` como identidad; la aritmética pasa a ser lo que es, la estructura de semianillo sobre ese vértice. `Nat.card` y `Set.Countable` se quedan como evidencia.

Un fallo del índice que sale gratis arreglar

Ya tienes lista de tokens genéricos descartados —`order` (515), `limit` (591), `continuous` (340)—. Le faltan estos, y cada uno viene probado por una ficha de esta misma hoja:

token	la ficha que lo prueba
theory	las mismas tres filas <code>FirstOrder.Language.*</code> aparecen en <code>homotopy-type-theory</code> , <code>ramsey-theory</code> , <code>computability-theory</code> y <code>proof-theory</code>
form	<code>canonical-forms</code> recibe <code>BilinForm</code> , <code>traceForm</code> , <code>killingForm</code> , <code>formPerm</code>
complete	<code>np-completeness</code> recibe <code>CompleteSpace</code> , <code>CompleteLattice</code> , <code>CauSeq.IsComplete</code>
discrete	<code>discrete-optimization</code> recibe <code>DiscreteTopology</code> , <code>CategoryTheory.Discrete</code>
analytic	<code>analytic-number-theory</code> recibe <code>AnalyticAt</code> , <code>AnalyticOnNhd</code> , <code>AnalyticOn</code>

Y una corrección de cita: el veredicto de `contour-integration` decía `Complex.integral_circle`. El nombre real es `circleIntegral`, en `MeasureTheory/Integral/CircleIntegral.lean`.

Orden de ejecución, por coste

1. **Gratis, sin medir** —no tocan el prompt—: las dos fusiones y retiradas (`sequent-calculus`, `recursion-theory`), el movimiento de `zfc-axioms` y `lean-kernel` a la capa de áreas, las 33 ramas declaradas, y los seis tokens genéricos.
2. **Barato, quitando**: los nombres de `limits-continuity` al campo de evidencia, el renombrado de `cardinal-arithmetic`, y la tanda de retirada medida de los siete.
3. **Caro, añadiendo** —una tanda aparte, con keyword declarada y midiendo su barrio—: `IsBrownianReal` y los tres hijos nuevos. Cuatro nombres en total, todos con token distintivo (`brownian`, `turing`, `extremal`, `matching`), que es la condición que falló la primera vez con `different`.

De las 48 fichas, una añade nombre por caducidad, tres añaden nodo, dos desaparecen, dos cambian de capa, dos se resuelven en C y treinta y tres se cierran de una vez con el campo `rol`.